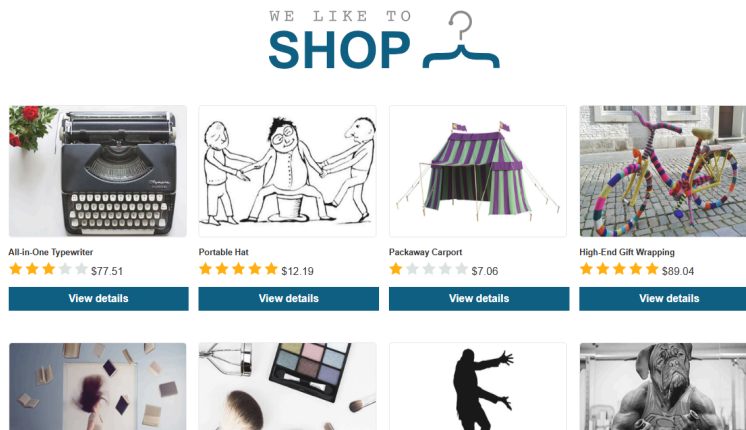




Actividad 09

File path traversal, traversal sequences stripped non-recursively

Guerrero Hurtado Lizbeth

Marco Teórico

¿Qué es File Path Traversal?

Es una vulnerabilidad de seguridad que ocurre cuando una aplicación web utiliza entradas proporcionadas por el usuario para construir rutas del sistema de archivos sin la debida validación. Permite a un atacante navegar por la estructura de directorios del servidor y acceder a archivos arbitrarios fuera del directorio web público (webroot).

Traversal Sequences Stripped Non-Recursively

Para mitigar ataques de navegación, algunos desarrolladores intentan aplicar un filtro que busca y elimina la secuencia de escape `../` del parámetro de entrada (por ejemplo, mediante funciones de reemplazo de cadenas como `strip()` o `replace("../", "")`). Sin embargo, si este proceso de eliminación se realiza de forma no recursiva (es decir, en una sola pasada sobre el texto), el filtro elimina la coincidencia exacta pero vuelve a ensamblar el texto restante. Esto deja intactas las secuencias `../` que hayan sido anidadas dentro del payload.

Mecanismo de Explotación

El fallo conceptual radica en cómo la función de sanitización procesa la cadena. Si el atacante envía la secuencia anidada `....//`, la función de filtrado detecta el patrón `../` en el centro de la cadena y lo elimina. Al borrar esos caracteres intermedios, las partes externas (`...` y `/`) se unen automáticamente, reconstruyendo de forma dinámica la secuencia prohibida `../`:

Payload enviado: `....//` → Filtrado (elimina central): `....//` → Resultado final: `../`

Punto Clave de Seguridad

Las estrategias de sanitización basadas en el borrado o reemplazo simple de cadenas son intrínsecamente inseguras. Si el motor de la aplicación no reevalúa la cadena procesada iterativamente hasta que esté libre de patrones maliciosos, la eliminación en una sola pasada genera un bypass directo por reconstrucción de payload.

Impacto en la Tríada CIA

Confidencialidad (Alto): Es el pilar directamente afectado. Un atacante puede leer archivos críticos del sistema como `/etc/passwd`, archivos de configuración con credenciales en texto plano (`.env`, `config.php`), claves privadas SSH y código fuente de la aplicación.

Integridad (Bajo): En este tipo de laboratorios no hay escritura directa, pero la exposición de código o credenciales de base de datos podría usarse en etapas posteriores para alterar datos en el sistema.

Disponibilidad (Medio): Acceder recursivamente a archivos pesados o dispositivos virtuales del sistema (/dev/urandom, /dev/zero) puede consumir recursos excesivos del servidor y derivar en un ataque de Denegación de Servicio (DoS).

Procedimiento de Explotación (Metodología en Burp Suite)

Reconocimiento y Captura del Tráfico:

- Abrir el navegador integrado de Burp Suite y acceder al sitio web del laboratorio.
- Navegar por la página de inicio y hacer clic en cualquier producto para que la aplicación cargue una imagen.
- Verificar en la pestaña Proxy > HTTP history la solicitud enviada para obtener el recurso multimedia, identificada como:

GET /image?filename=54.jpg HTTP/1.1

Transferencia a Repeater:

- Hacer clic derecho sobre la solicitud del parámetro filename y seleccionar Send to Repeater (o presionar Ctrl + R).

Prueba de Filtro Inicial (Comprobación del Bloqueo):

- En la pestaña Repeater, intentar un payload estándar de Path Traversal: ?filename=../../etc/passwd.
- Enviar la petición y observar que la aplicación devuelve un error o un recurso no encontrado, lo que confirma la presencia de un filtro que está stripping/eliminando los caracteres ../.

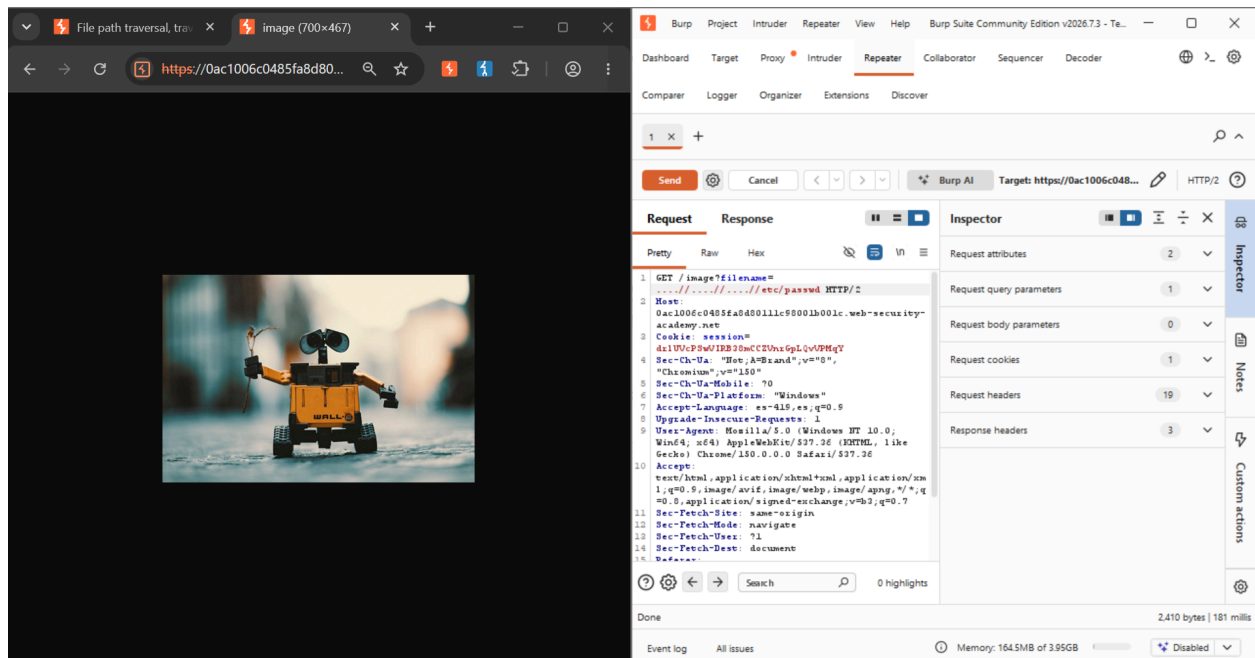
Inyección del Payload Anidado (Bypass No Recursivo):

- Reemplazar cada secuencia ../ por el patrón anidado//.
- Modificar la línea de la petición en Repeater para que quede estructurada así:

GET /image?filename=....//....//....//etc/passwd HTTP/1.1

Exfiltración y Confirmación:

- Hacer clic en Send.
- Verificar que la respuesta del servidor sea un código HTTP/2 200 OK.
- Confirmar en el cuerpo de la respuesta (Response Body) el archivo /etc/passwd con las cuentas de usuario exfiltradas.
- Volver al navegador para validar la resolución completa con el banner de "Lab Solved".



Estrategias de Mitigación Recomendadas

Reemplazar la Sanitización por Listas Blancas (Whitelisting):

Evitar limpiar o modificar entradas sospechosas. En su lugar, validar estrictamente que la entrada del usuario cumpla con un formato seguro mediante expresiones regulares alfanuméricas:

`^[a-zA-Z0-9_-]+\.(png|jpg|jpeg)$`

Uso de Claves e Índices Indirectos:

No pasar nombres de archivos ni rutas a través de parámetros URL. Se debe usar un identificador numérico o mapeo interno (ej. GET /image?id=42), gestionando las rutas reales únicamente desde el backend.

Validación de Ruta Canónica (Canonicalization):

Antes de abrir el archivo en el servidor, resolver la ruta canónica y verificar de forma programática que empiece dentro del directorio base permitido.

Ejemplo en Java:

```
File file = new File(BASE_DIRECTORY, userInput);

String canonicalPath = file.getCanonicalPath();

if (!canonicalPath.startsWith(BASE_DIRECTORY)) {

    throw new SecurityException("Intento de Path Traversal detectado.");

}
```

Principio de Menor Privilegio en el Sistema Operativo:

Ejecutar el proceso del servidor web (www-data, nobody, etc.) con los permisos mínimos necesarios en el sistema de archivos, impidiendo que el usuario tenga acceso de lectura a /etc/passwd u otros directorios del sistema.

Conclusiones y Aprendizaje

- Peligro de los Filtros Simplistas: Este laboratorio demuestra que intentar "limpiar" la entrada del usuario borrando caracteres peligrosos mediante una sola pasada es una práctica insegura, ya que las cadenas resultantes pueden recombinarse de forma imprevista.
- Inseguridad de las Listas Negras: Las defensas basadas en buscar y remover patrones específicos siempre pueden ser evadidas con técnicas de anidamiento, codificación u omisiones de lógica.
- Enfoque de Seguridad por Diseño: La prevención efectiva de Path Traversal no requiere filtros más complejos sobre el texto, sino aplicar validación por listas blancas, resolución canónica de rutas y el principio de menor privilegio a nivel de servidor.